

Plan de développement MVP — GarageFlow

1. Objectif du plan

Le but du plan de développement est de construire le MVP GarageFlow dans un ordre logique, sans se perdre dans les fonctionnalités secondaires.

Le MVP doit permettre de démontrer le parcours suivant :

1. Un client se connecte.
2. Il ajoute un véhicule.
3. Il consulte un garage.
4. Il choisit une prestation.
5. Il choisit un créneau.
6. Il envoie une demande de rendez-vous.
7. Le garage se connecte à son dashboard.
8. Le garage accepte le rendez-vous.
9. Une intervention est créée.
10. Le garage change le statut de l'intervention.
11. Le client suit l'avancement depuis son mobile.
12. Le client reçoit une notification.

Ce parcours permet de couvrir les attentes principales de l'évaluation : application web, application mobile, backend sécurisé, base de données, UX, sécurité et documentation.

2. Stratégie générale de développement

Le développement doit se faire dans cet ordre :

1. socle projet ;
2. base de données ;
3. authentification ;
4. configuration garage ;
5. réservation client ;
6. dashboard garage ;
7. intervention et statuts ;
8. notifications ;
9. interface mobile ;
10. tests, polish et documentation.

Le backend doit être prioritaire, car il sert de base à l'application web et à l'application mobile.

3. Organisation Git recommandée

Branches principales

```
main  
develop
```

Branches de fonctionnalités

```
feature/project-setup  
feature/backend-auth  
feature/backend-database  
feature/garage-settings  
feature/client-vehicles  
feature/appointments  
feature/garage-dashboard  
feature/interventions  
feature/notifications  
feature/web-ui  
feature/mobile-ui  
feature/tests  
feature/docs
```

Convention de commits

```
feat(auth): add JWT login endpoint  
feat(vehicles): add client vehicle creation  
feat(appointments): create appointment request  
feat(garage): add appointment validation  
feat(interventions): update intervention status  
fix(security): restrict garage data access  
docs(mcd): add data model documentation  
docs(api): document appointment endpoints
```

4. Sprint 0 — Initialisation du projet

Objectif

Créer une base de projet propre avant de développer les fonctionnalités.

Tâches

GitHub

- Créer le repository GitHub.
- Créer les branches `main` et `develop`.
- Ajouter un README.
- Ajouter un `.gitignore`.
- Créer le dossier `/docs`.
- Créer les premières issues GitHub.

Backend Symfony

- Créer le projet Symfony dans `/backend`.
- Configurer l'environnement local.
- Configurer MySQL.
- Installer Doctrine ORM.
- Installer Doctrine Migrations.
- Préparer le fichier `.env.example`.

Web React

- Créer le projet React + Vite dans `/web`.
- Installer Tailwind CSS.
- Créer une structure de dossiers propre.
- Créer une page de test.

Mobile Expo

- Créer le projet React Native Expo dans `/mobile`.
- Créer la navigation de base.
- Créer un écran de test.

Documentation

- Créer la documentation fonctionnelle.
- Créer la documentation technique.
- Ajouter le MCD, MLD et dictionnaire de données.

Résultat attendu

Le projet est prêt à être développé avec une structure propre.

5. Sprint 1 — Base de données et entités backend

Objectif

Créer les entités principales et générer la base de données.

Tâches backend

Créer les entités Doctrine :

- Role ;
- User ;
- Garage ;
- OpeningHour ;
- Unavailability ;
- Service ;
- Vehicle ;
- Appointment ;
- InterventionStatus ;
- Intervention ;
- InterventionStatusHistory ;
- InternalNote ;
- Notification ;
- ActionLog.

Créer les migrations Doctrine.

Exécuter les migrations.

Ajouter les données de référence :

- rôles ;
- statuts d'intervention.

Tâches documentation

- Ajouter la capture du MCD.
- Ajouter le MLD.
- Ajouter le dictionnaire de données.
- Expliquer le rôle des tables principales.

Résultat attendu

La base de données est créée et cohérente avec le MCD/MLD.

6. Sprint 2 — Authentification et sécurité

Objectif

Permettre aux utilisateurs de se connecter de manière sécurisée.

Tâches backend

- Créer l'inscription client.
- Créer la connexion.
- Mettre en place JWT.
- Hasher les mots de passe.
- Créer l'endpoint `/api/auth/me`.
- Protéger les routes privées.
- Mettre en place les rôles :
 - `ROLE_ADMIN`
 - `ROLE_GERANT`
 - `ROLE_EMPLOYE`
 - `ROLE_CLIENT`

Tâches web

- Créer l'écran de connexion garage.
- Stocker le token après connexion.
- Rediriger vers le dashboard garage.

Tâches mobile

- Créer l'écran de connexion client.
- Créer l'écran d'inscription client.
- Stocker le token côté mobile.
- Rediriger vers l'accueil mobile.

Tests à prévoir

- Un utilisateur peut se connecter.
- Un mauvais mot de passe affiche une erreur.
- Une route protégée refuse un utilisateur non connecté.
- Un client ne peut pas accéder à une route garage.

Résultat attendu

L'application possède un socle de sécurité fonctionnel.

7. Sprint 3 — Configuration garage

Objectif

Permettre au garage de configurer ses prestations, horaires et indisponibilités.

Tâches backend

Créer les endpoints :

```
GET /api/garage/services
POST /api/garage/services
PATCH /api/garage/services/{id}
DELETE /api/garage/services/{id}

GET /api/garage/opening-hours
POST /api/garage/opening-hours
PATCH /api/garage/opening-hours/{id}
DELETE /api/garage/opening-hours/{id}

GET /api/garage/unavailabilities
POST /api/garage/unavailabilities
DELETE /api/garage/unavailabilities/{id}
```

Tâches web

Créer les pages :

- gestion des prestations ;
- gestion des horaires ;
- gestion des indisponibilités.

Règles métier

- Une prestation appartient à un seul garage.
- Une prestation doit avoir une durée.
- Un horaire doit avoir une heure de début inférieure à l'heure de fin.
- Une indisponibilité bloque un créneau ou une journée.
- Un garage ne peut modifier que ses propres données.

Résultat attendu

Le garage peut configurer les données nécessaires à la réservation.

8. Sprint 4 — Véhicules client

Objectif

Permettre au client d'ajouter et gérer ses véhicules.

Tâches backend

Créer les endpoints :

```
GET /api/client/vehicles
POST /api/client/vehicles
PATCH /api/client/vehicles/{id}
DELETE /api/client/vehicles/{id}
```

Tâches mobile

Créer les écrans :

- mes véhicules ;
- ajouter un véhicule ;
- modifier un véhicule ;
- sélectionner un véhicule lors de la réservation.

Règles métier

- Un client peut avoir plusieurs véhicules.
- Un véhicule appartient à un seul client.
- Un client ne peut voir que ses propres véhicules.
- Un garage ne peut voir un véhicule que s'il est lié à un rendez-vous de son garage.

Résultat attendu

Le client peut préparer son parcours de réservation avec un véhicule enregistré.

9. Sprint 5 — Recherche garage et parcours de réservation mobile

Objectif

Créer le parcours mobile principal : chercher un garage, choisir une prestation, choisir un créneau et envoyer une demande.

Tâches backend

Créer les endpoints :

```
GET /api/garages
GET /api/garages/{id}
GET /api/garages/{id}/services
GET /api/garages/{id}/available-slots
POST /api/client/appointments
```

Créer un service métier :

AvailabilityService

Responsabilités :

- calculer les créneaux disponibles ;
- vérifier les horaires du garage ;
- vérifier les indisponibilités ;
- vérifier les rendez-vous déjà existants ;
- empêcher les doubles réservations.

Créer un service métier :

AppointmentService

Responsabilités :

- vérifier que le client est connecté ;
- vérifier que le véhicule appartient au client ;
- vérifier que la prestation appartient au garage ;
- créer une demande de rendez-vous au statut `EN_ATTENTE` .

Tâches mobile

Créer les écrans :

- liste des garages ;
- détail garage ;
- choix prestation ;
- choix créneau ;
- choix véhicule ;
- confirmation de demande de rendez-vous.

Règles métier

- Le client ne peut pas réserver dans le passé.

- Le client ne peut pas réserver un créneau indisponible.
- Le client choisit une seule prestation.
- Le rendez-vous est créé au statut `EN_ATTENTE`.
- Le garage doit ensuite accepter ou refuser.

Résultat attendu

Un client peut créer une demande de rendez-vous depuis le mobile.

10. Sprint 6 — Dashboard garage et gestion des rendez-vous

Objectif

Permettre au garage de gérer les demandes de rendez-vous.

Tâches backend

Créer les endpoints :

```
GET /api/garage/dashboard
GET /api/garage/appointments
GET /api/garage/appointments/{id}
PATCH /api/garage/appointments/{id}/accept
PATCH /api/garage/appointments/{id}/refuse
PATCH /api/appointments/{id}/cancel
```

Créer ou compléter :

```
AppointmentService
InterventionService
NotificationService
LogActionService
```

Lorsqu'un rendez-vous est accepté :

- le statut du rendez-vous passe à `CONFIRME` ;
- une intervention est créée ;
- un premier statut d'intervention est ajouté ;
- une notification est créée ;
- une action est journalisée.

Tâches web

Créer les pages :

- dashboard ;
- demandes de rendez-vous ;
- détail rendez-vous ;
- planning simple.

Règles métier

- Un employé ou gérant ne voit que les rendez-vous de son garage.
- Le garage peut accepter ou refuser une demande.
- Un rendez-vous accepté déclenche une intervention.
- Un rendez-vous refusé ne déclenche pas d'intervention.

Résultat attendu

Le garage peut traiter les demandes de rendez-vous depuis son dashboard web.

11. Sprint 7 — Interventions et suivi de statut

Objectif

Permettre au garage de suivre une intervention et au client de voir l'avancement.

Tâches backend

Créer les endpoints :

```
GET /api/garage/interventions
GET /api/garage/interventions/{id}
PATCH /api/garage/interventions/{id}/status
POST /api/garage/interventions/{id}/notes

GET /api/client/interventions
GET /api/client/interventions/{id}
```

Créer ou compléter :

```
InterventionService
InterventionStatusHistoryService
NotificationService
```

Tâches web

Créer les pages :

- liste des interventions ;
- détail intervention ;
- changement de statut ;
- ajout note interne.

Tâches mobile

Créer les écrans :

- mes rendez-vous ;
- mes interventions en cours ;
- détail intervention ;
- timeline des statuts.

Règles métier

- Chaque changement de statut est historisé.
- Le client voit seulement les informations autorisées.
- Le client ne voit jamais les notes internes.
- Le changement de statut peut créer une notification.
- Le statut “véhicule prêt” déclenche une notification importante.

Résultat attendu

Le garage peut mettre à jour l'avancement du véhicule et le client peut suivre le statut.

12. Sprint 8 — Notifications

Objectif

Informier le client des événements importants.

Tâches backend

Créer les endpoints :

```
GET /api/notifications
PATCH /api/notifications/{id}/read
```

Créer ou compléter :

NotificationService

Types de notifications MVP :

- rendez-vous demandé ;
- rendez-vous accepté ;
- rendez-vous refusé ;
- rendez-vous annulé ;
- changement de statut ;
- véhicule prêt.

Tâches mobile

Créer les écrans :

- liste des notifications ;
- détail notification ;
- filtre lu / non lu.

Tâches web

Afficher éventuellement un indicateur de notifications côté garage si utile.

Règles métier

- Une notification appartient à un destinataire.
- Un utilisateur ne peut voir que ses notifications.
- Une notification peut être liée à un rendez-vous ou à une intervention.

Résultat attendu

Le client reçoit et consulte les notifications liées à son rendez-vous ou son intervention.

13. Sprint 9 — Polish UI/UX, accessibilité et performance

Objectif

Améliorer la qualité perçue du projet avant l'oral.

Tâches UI/UX

- Ajouter les messages d'erreur.
- Ajouter les messages de succès.
- Ajouter les loaders.

- Ajouter les confirmations avant actions sensibles.
- Harmoniser les statuts avec des badges.
- Simplifier les textes.
- Corriger les libellés ambigus.

Tâches accessibilité

- Vérifier les contrastes.
- Ajouter des labels aux champs.
- Vérifier la taille des boutons.
- Prévoir des messages d'erreur compréhensibles.
- Éviter les textes trop petits.

Tâches performance

- Ajouter pagination ou filtres sur les listes longues.
- Limiter les données renvoyées par l'API.
- Ajouter les index SQL principaux.
- Éviter de charger tout le planning inutilement.
- Séparer les appels dashboard, planning et interventions.

Résultat attendu

L'application devient plus propre, plus claire et plus défendable à l'oral.

14. Sprint 10 — Tests et documentation finale

Objectif

Préparer le rendu final et l'oral.

Tests fonctionnels

Scénarios à tester :

- inscription client ;
- connexion client ;
- ajout véhicule ;
- recherche garage ;
- demande de rendez-vous ;
- connexion garage ;
- acceptation rendez-vous ;
- création intervention ;
- changement statut ;
- notification client ;
- consultation suivi intervention.

Tests de sécurité

Scénarios à tester :

- un client ne peut pas voir les véhicules d'un autre client ;
- un client ne peut pas accéder au dashboard garage ;
- un garage ne peut pas voir les rendez-vous d'un autre garage ;
- un client ne peut pas voir les notes internes ;
- une route protégée refuse un utilisateur non connecté.

Documentation finale

À finaliser :

- cahier des charges ;
- MCD ;
- MLD ;
- dictionnaire de données ;
- matrice des droits ;
- architecture technique ;
- parcours utilisateurs et maquettes ;
- documentation API ;
- guide d'installation ;
- guide de démonstration ;
- support oral.

Résultat attendu

Le projet est prêt pour la démonstration finale.

15. Issues GitHub recommandées

Sprint 0 — Setup

- SETUP-001 — Créer le repository GitHub
- SETUP-002 — Initialiser Symfony API
- SETUP-003 — Initialiser React web
- SETUP-004 — Initialiser Expo mobile
- SETUP-005 — Créer la structure docs

Sprint 1 — Database

- DB-001 — Créer les entités Doctrine
- DB-002 — Créer les migrations
- DB-003 — Ajouter les rôles
- DB-004 — Ajouter les statuts d'intervention
- DB-005 — Ajouter les index principaux

Sprint 2 — Auth

- AUTH-001 — Inscription client
- AUTH-002 — Connexion JWT
- AUTH-003 — Endpoint utilisateur connecté
- AUTH-004 — Protection des routes
- AUTH-005 — Redirection selon rôle

Sprint 3 — Garage settings

- GARAGE-001 — CRUD prestations
- GARAGE-002 — CRUD horaires
- GARAGE-003 — CRUD indisponibilités
- GARAGE-004 — Sécurité isolation garage

Sprint 4 — Vehicles

- VEHICLE-001 — Liste véhicules client
- VEHICLE-002 — Ajout véhicule
- VEHICLE-003 — Modification véhicule
- VEHICLE-004 — Suppression véhicule
- VEHICLE-005 — Sélection véhicule réservation

Sprint 5 — Appointments mobile

- RDV-001 — Liste garages actifs
- RDV-002 — Détail garage
- RDV-003 — Liste prestations garage
- RDV-004 — Calcul créneaux disponibles
- RDV-005 — Création demande rendez-vous
- RDV-006 — Confirmation mobile

Sprint 6 — Garage dashboard

- DASH-001 — Dashboard garage
- DASH-002 — Liste demandes en attente
- DASH-003 — Détail rendez-vous
- DASH-004 — Accepter rendez-vous
- DASH-005 — Refuser rendez-vous
- DASH-006 — Planning garage

Sprint 7 — Interventions

- INT-001 — Création intervention après acceptation
- INT-002 — Liste interventions garage
- INT-003 — Détail intervention garage
- INT-004 — Changement statut
- INT-005 — Historique statuts
- INT-006 — Notes internes
- INT-007 — Suivi intervention mobile

Sprint 8 — Notifications

- NOTIF-001 — Créer notification rendez-vous accepté
- NOTIF-002 — Créer notification changement statut
- NOTIF-003 — Liste notifications client
- NOTIF-004 — Marquer notification comme lue

Sprint 9 — Qualité

- UX-001 — Ajouter loaders
- UX-002 — Ajouter messages succès / erreur
- UX-003 — Corriger labels et textes
- UX-004 — Vérifier responsive web
- UX-005 — Vérifier accessibilité basique

Sprint 10 — Rendu

- DOC-001 — Documentation API
 - DOC-002 — Guide installation
 - DOC-003 — Scénario de démo
 - DOC-004 — Captures écrans
 - DOC-005 — Support oral final
-

16. Ordre de développement prioritaire

Priorité absolue

1. Backend Symfony
2. Base de données
3. Authentification JWT
4. Gestion véhicules
5. Gestion prestations / horaires
6. Création rendez-vous
7. Dashboard garage
8. Acceptation rendez-vous
9. Intervention
10. Statuts
11. Notifications
12. Mobile client
13. Polish UI/UX
14. Documentation finale

Pourquoi cet ordre ?

Le backend et la base de données doivent arriver en premier, car le web et le mobile dépendent de l'API.

La réservation doit arriver avant le dashboard garage, car le garage doit avoir des demandes à traiter.

L'intervention doit arriver après l'acceptation du rendez-vous, car elle dépend d'un rendez-vous confirmé.

Les notifications arrivent après les rendez-vous et statuts, car elles dépendent des événements métier.

17. Scope MVP verrouillé

Inclus dans le MVP

- Authentification
- Rôles
- Garage
- Prestations
- Horaires
- Indisponibilités simples
- Véhicules
- Rendez-vous
- Acceptation / refus
- Intervention
- Statuts
- Historique des statuts
- Notes internes
- Notifications app
- Dashboard garage
- Application mobile client

Exclu du MVP

- Paiement
- Facturation
- Stock pièces
- Devis complexe
- Chat temps réel
- SMS
- Documents avancés
- Favoris
- Avis clients
- IA
- Multi-langue
- Marketplace

Ces fonctionnalités peuvent être présentées comme évolutions futures.

18. Scénario de démonstration final

Démo côté client

1. Connexion du client.
2. Ajout ou sélection d'un véhicule.
3. Recherche d'un garage.
4. Consultation de la fiche garage.
5. Sélection d'une prestation.
6. Choix d'un créneau.
7. Envoi de la demande de rendez-vous.

Démo côté garage

1. Connexion du gérant.
2. Ouverture du dashboard.
3. Consultation des demandes en attente.
4. Acceptation du rendez-vous.
5. Création automatique d'une intervention.
6. Changement du statut de l'intervention.
7. Ajout d'une note interne.

Retour côté client

1. Consultation des notifications.
2. Consultation de l'intervention.
3. Affichage du nouveau statut.
4. Visualisation de l'historique.

Ce scénario est court, clair et couvre tout le cœur du projet.

19. Synthèse orale

Le développement a été organisé autour du parcours métier principal : un client prend rendez-vous, le garage valide la demande, puis suit l'intervention jusqu'à la récupération du véhicule.

J'ai priorisé le backend et la base de données afin de sécuriser les règles métier avant de développer les interfaces. L'application web garage et l'application mobile client consomment ensuite la même API Symfony.

Le découpage en sprints permet de relier chaque étape de développement aux attendus de l'évaluation : base de données, backend sécurisé, application web, application mobile, UX, GitHub, tests et documentation.